Machine Learning Lectures

Classification part 1

Classification – Part 1

By:
Dr. Andrey Holz
https://www.linkedin.com/in/andrey-holz

Machine Learning Lectures

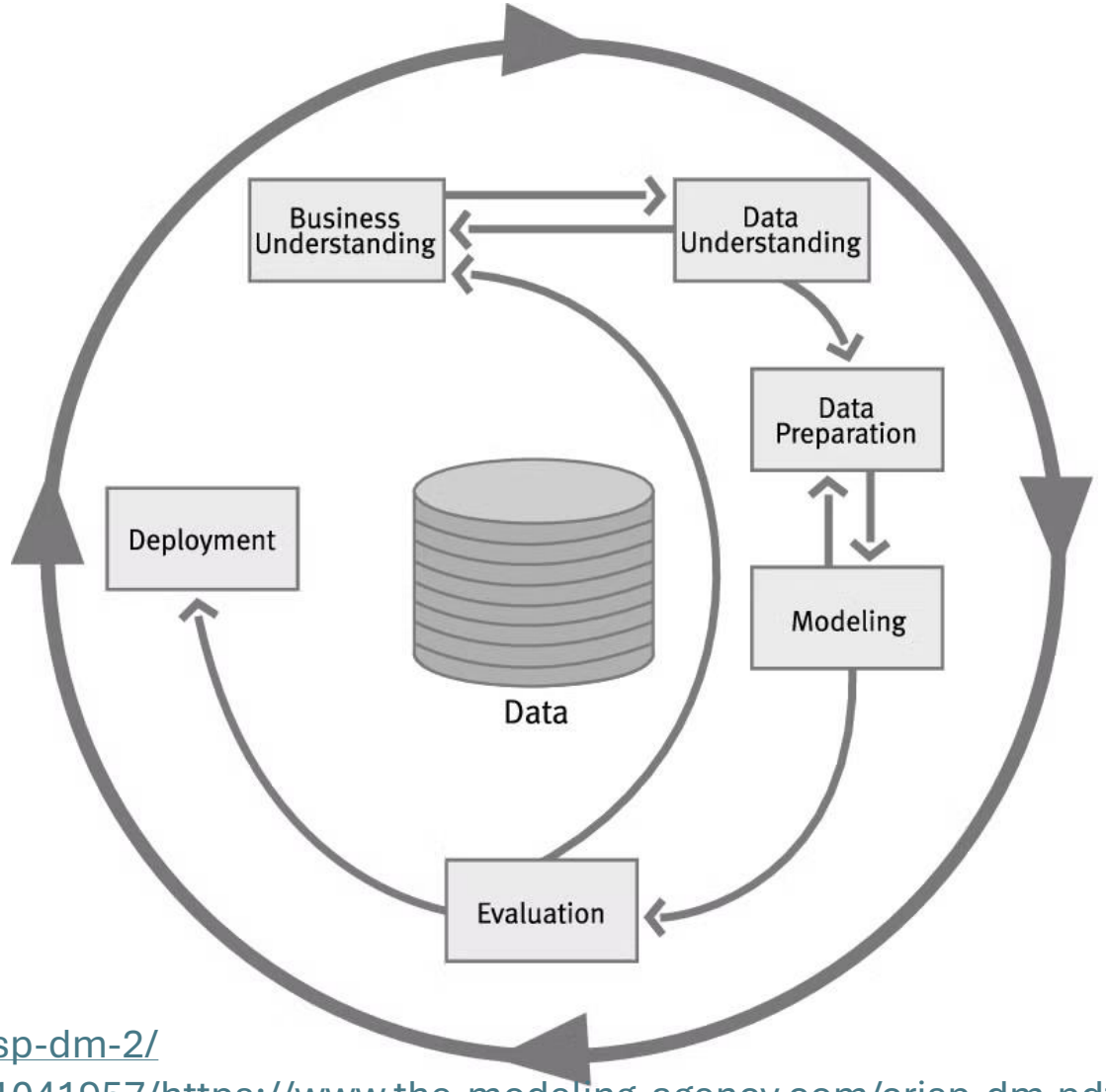
Classification – Part 1

By:
Dr. Andrey Holz
https://www.linkedin.com/in/andrey-holz

Agenda

1. Classification Problem Setting
2. Basic Classification Models:
    1. K Nearest Neighbors
    2. Decision Tree Classifier
    3. Logistic Regression
    4. Support Vector Classifier
    5. Naive Bayes
3. Model Evaluation and Validation
4. Challenges
5. Key Notes

Data Analytics pipeline: CRISP-DM



https://www.datascience-pm.com/crisp-dm-2/
https://web.archive.org/web/20220401041957/https://www.the-modeling-agency.com/crisp-dm.pdf

Business Understanding

Determine Business Objectives
Background
Business Objectives
Business Success Criteria

Assess Situation
Inventory of Resources
Requirements, Assumptions, and Constraints
Risks and Contingencies
Terminology
Costs and Benefits

Determine Data Mining Goals
Data Mining Goals
Data Mining Success Criteria

Produce Project Plan
Project Plan
Initial Assessment of Tools and Techniques

Data Understanding

Collect Initial Data
Initial Data Collection Report

Describe Data
Data Description Report

Explore Data
Data Exploration Report

Verify Data Quality
Data Quality Report

Data Preparation

Select Data
Rationale for Inclusion/ Exclusion

Clean Data
Data Cleaning Report

Construct Data
Derived Attributes
Generated Records

Integrate Data
Merged Data

Format Data
Reformatted Data

Dataset
Dataset Description

Modeling

Select Modeling Techniques
Modeling Technique
Modeling Assumptions

Generate Test Design
Test Design

Build Model
Parameter Settings
Models
Model Descriptions

Assess Model
Model Assessment
Revised Parameter Settings

Evaluation

Evaluate Results
Assessment of Data Mining Results w.r.t. Business Success Criteria
Approved Models

Review Process
Review of Process

Determine Next Steps
List of Possible Actions
Decision

Deployment

Plan Deployment
Deployment Plan

Plan Monitoring and Maintenance
Monitoring and Maintenance Plan

Produce Final Report
Final Report
Final Presentation

Review Project
Experience Documentation

Introduction to Classification

Classification Problem

Classification assigns labels or categories (**target**) to new observations based on historical data patterns (**features**). It's a **supervised** ML problem.

Real-World Examples:

- Spam Detection ,
- Image Recognition,
- Disease Diagnosis
- Antifraud

Types of Classification:

- **Binary** - Two possible outcomes
- **Multiclass** - More than two categories
- **Multilabel** - Multiple labels can apply simultaneously
- **Ordinal** - Categories have a natural order

Classification Problem

Suppose we have a dataset of pairs:

$$\{(X_i, Y_i)\}_{i=1}^{m}$$

where:

$$X_i \in \mathbb{R}^n$$

represents the i -th observation of n-dimensional features, and

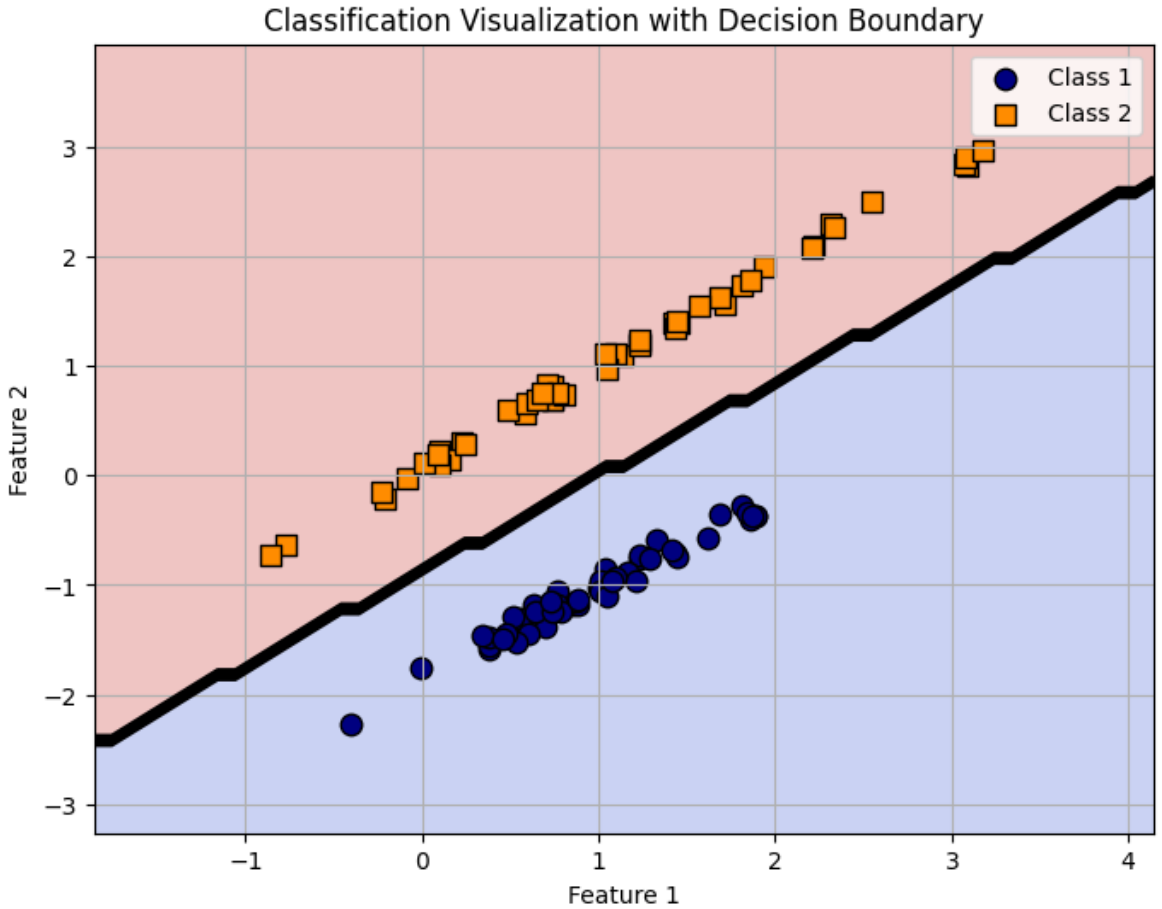
$$Y_i \in C$$

is the corresponding class label from a set of k possible classes $\{1, 2 \ldots k\}$.

The goal is to approximate the unknown relationship:

$$Y = f(X)$$

to classify new instances accurately into one of the k classes



Basic Classification Models

K-Nearest Neighbors (KNN)

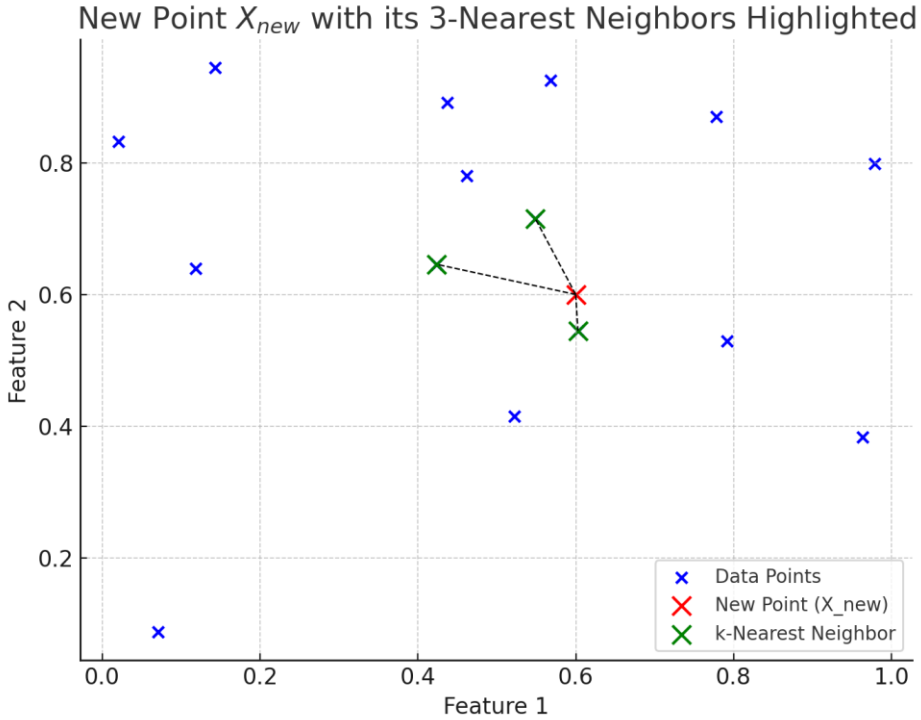
- **Definition:** KNN is a **non-parametric supervised** algorithm that classifies a new point based on the majority class of its *k* nearest neighbors
- **Mathematical Foundation**:

$$Y_{new} = argmax_{c \in C} \sum_{i \in N_k(X_{new})} \mathbb{I}(Y_i = c)$$

where:
$N_k(X_{new})$ – indexes of the **k**-nearest neighbors.
$\mathbb{I}$ - Indicator function (1 if condition is true, else 0).
C – class label, $\{1, 2 \ldots k\}$,.

Key Insight: KNN assigns the most frequent label among the nearest neighbors to the new data point.

Optimal k should be selected with Cross-Validation and GridSearch (or Elbow Method)



Distance Metrics in KNN

Euclidean Distance:

$$d(X_i, X_j) = \sqrt{\sum_{k=1}^{n} (X_{ik} - X_{jk})^2}$$

- **Use Case:** Most common, suitable for continuous variables.

Manhattan Distance:

$$d(X_i, X_j) = \sum_{k=1}^{n} |X_{ik} - X_{jk}|$$

- **Use Case:** Grid-like data structures.

Minkowski Distance (Generalized):

$$d(X_i, X_j) = \left(\sum_{k=1}^{n} |X_{ik} - X_{jk}|^p\right)^{\frac{1}{p}}$$

- p=2 yields Euclidean distance.
- p=1 yields Manhattan distance.

- **Cosine Distance:**

$$\text{Cosine Distance}(X_i, X_j) = 1 - \frac{X_i \cdot X_j}{\|X_i\| \|X_j\|}$$

- **Use Case:** High-dimensional spaces, such as text data.

$$X_i \cdot X_j = \sum_{k=1}^{n} X_{ik} X_{jk}$$

```
Example:
vectorA = [3,4,5]
vectorB = [1,0,7]
==Distance==
Euclidean:        4.899
Manhattan:        8.000
Minkowski (p=3): 4.309
Cosine:           0.240
```

Computational Complexity of KNN

Training Complexity: $O(1)$; *KNN* does not perform training; it stores the data.

Prediction Complexity: $O(m \times n)$; *m* - training samples; *n* - features.

Optimization Techniques:

- **KD-Trees**: Efficient for low-dimensional data by partitioning the feature space.
- **Approximate Nearest Neighbors (ANN)**: Locality-sensitive hashing for high-dimensional data.

Curse of Dimensionality: As the number of dimensions increases, the distance between points becomes less meaningful, leading to sparse data distributions.

- **Distance Concentration:** $\lim_{n \to \infty} d_{min}/d_{max} \approx 1$

  In high dimensions, the nearest and farthest neighbors converge in distance.

- **Reduced Discriminative Power**: Difficulty in distinguishing between classes.
- **Increased Sensitivity to Noise:** Sparse data makes KNN more susceptible to irrelevant features.

Applications and Limitations of KNN

Applications:

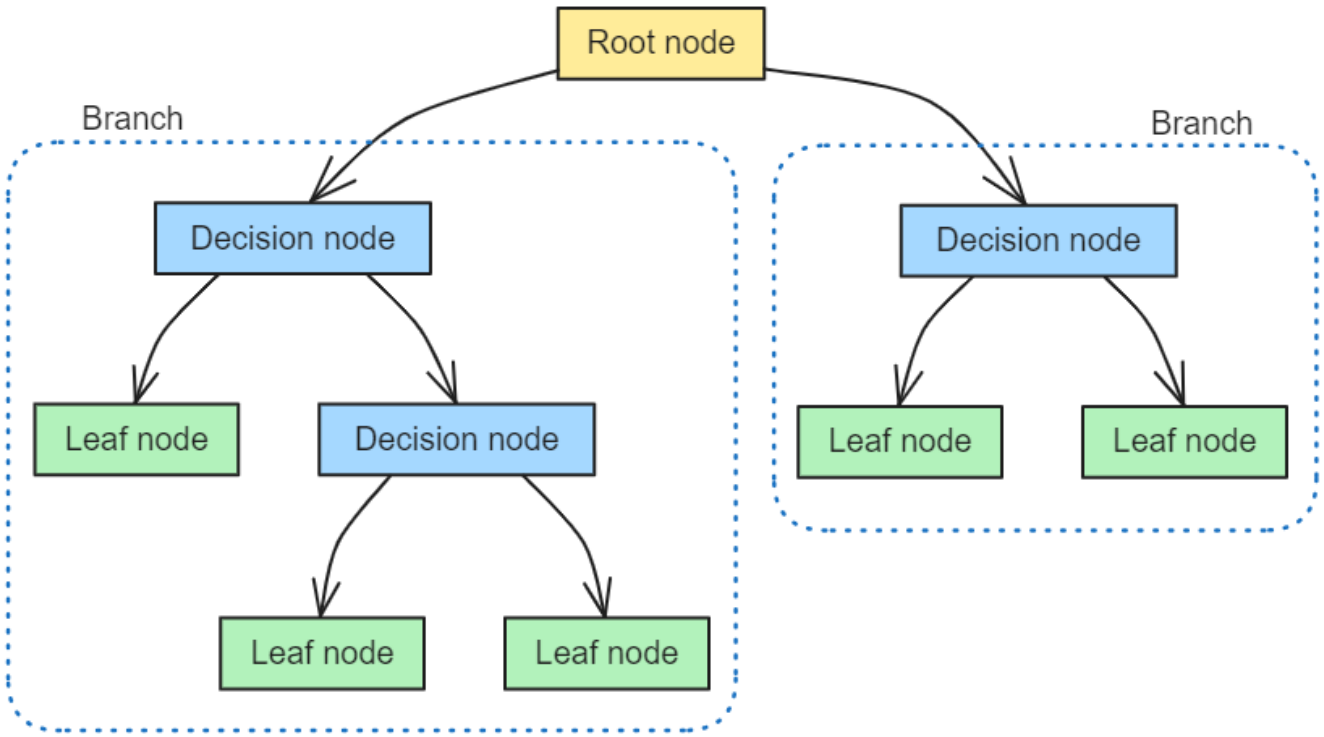
- 🖼️ **Image Classification:** Recognizing objects within images.
- 🛒 **Recommendation Systems:** Suggesting products based on user similarity.
- ⚠️ **Anomaly Detection:** Identifying unusual patterns or outliers.
- 🧬 **Biological Classification:** Categorizing species or genetic information.

Practical Limitations:

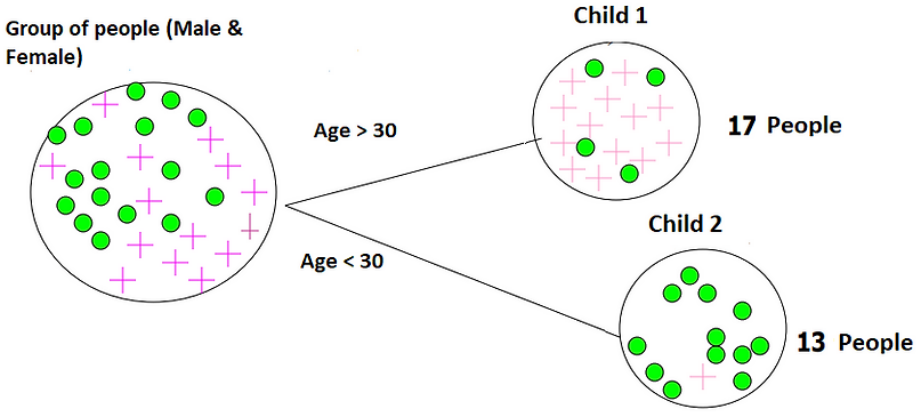
- ⚡ **Sensitivity to Noise:** Mislabelled or irrelevant data can degrade performance.
- 🌌 **Curse of Dimensionality:** Performance deteriorates as feature space grows.
- ⏳ **Computational Inefficiency:** High resource consumption with large datasets.

Live Coding & Demos

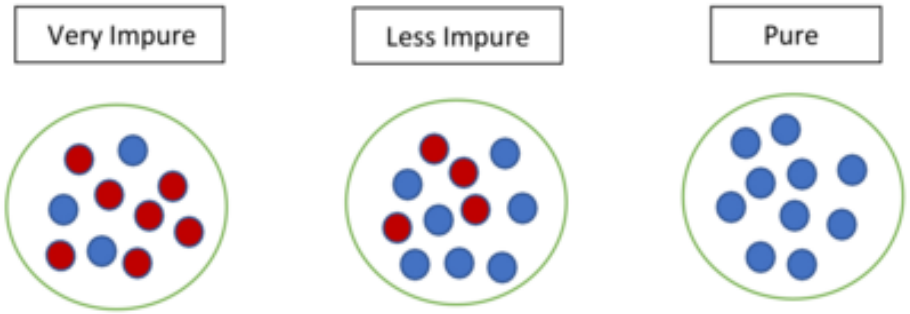
Decision Tree



Video Explanation:
https://www.youtube.com/watch?v=ZVR2Way4nwQ



https://www.numpyninja.com/post/what-is-entropy-and-information-gain-how-are-they-used-to-construct-decision-trees



Gini impurity and **entropy** are common metrics, favoring splits that best separate classes. **Information Gain** uses entropy reduction to find ideal splits.

Decision Tree Classifier

- **Gini Impurity:**
  - $Gini(D) = 1 - \sum_{c \in C} p(c)^2$

  where p(c) is the probability of class ccc in dataset D.
  Measures the probability of incorrect classification at a node.

- **Entropy** (Information Gain):
  - Measures the disorder at a node, with higher entropy indicating more mixed classes. For a node with classes *C*:

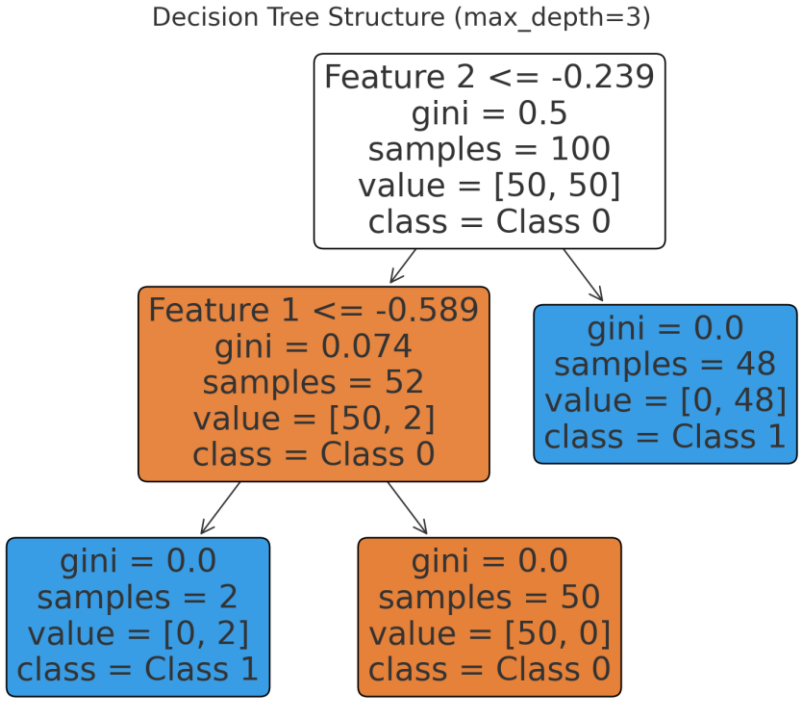
$$\mathrm{H}(D) = -\sum_{c \in C} p(c)\log(p(c))$$

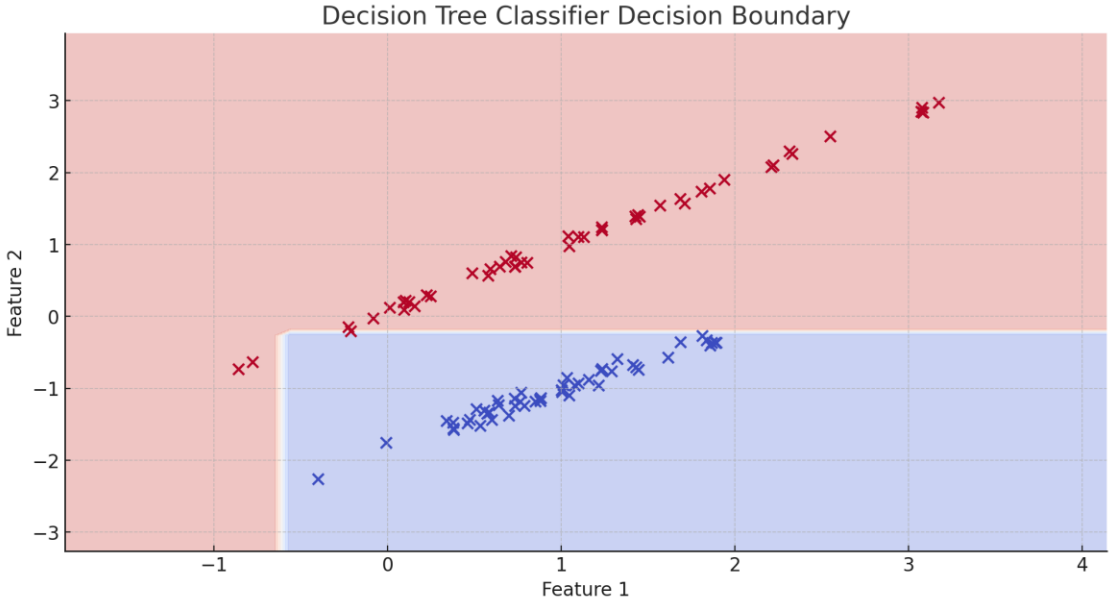
  - **Information Gain:** The reduction in entropy from a split

$$\mathrm{IG}(D, split) = \mathrm{H}(D) - \sum_{i=1}^{k} \frac{|D_i|}{|D|}\mathrm{H}(D_i)$$

  H(*D*): The entropy of the original dataset *D*, which represents the uncertainty in D before the split.
  $H(Di)$: The entropy of each subset Di after the split.
  $|Di|/|D|$: The proportion of samples in subset Di compared to the total samples in D. This weight ensures that larger subsets contribute more to the Information Gain calculation.

- Decision boundaries in decision trees are **axis-aligned** and appear as rectangular regions in the feature space due to binary splits on single features.
- For each split, a boundary line is introduced along the selected feature's axis, separating data based on threshold values.





Computational Complexity of Decision Trees

Training Complexity: $O(m \times n \times \log(m))$;
Each split examines all data points, and a balanced tree has a depth of $\log(m)$. For a well-balanced binary tree (where each node has two children until the leaves), the depth ***d*** relates to the number of nodes ***m*** as follows: $m \approx 2^d$
This means that, ideally, each level (or depth increment) doubles the number of nodes. $d \approx \log_2 m$ This represents the **depth required to split m nodes fully** in a balanced binary tree.

Prediction Complexity: $O(\log(m))$;
for a balanced tree, as each sample passes from root to leaf.

Overfitting and Complexity Control in Decision Trees

Overfitting Risk:
Decision trees are prone to overfitting, particularly with deep trees that capture noise in the training data.

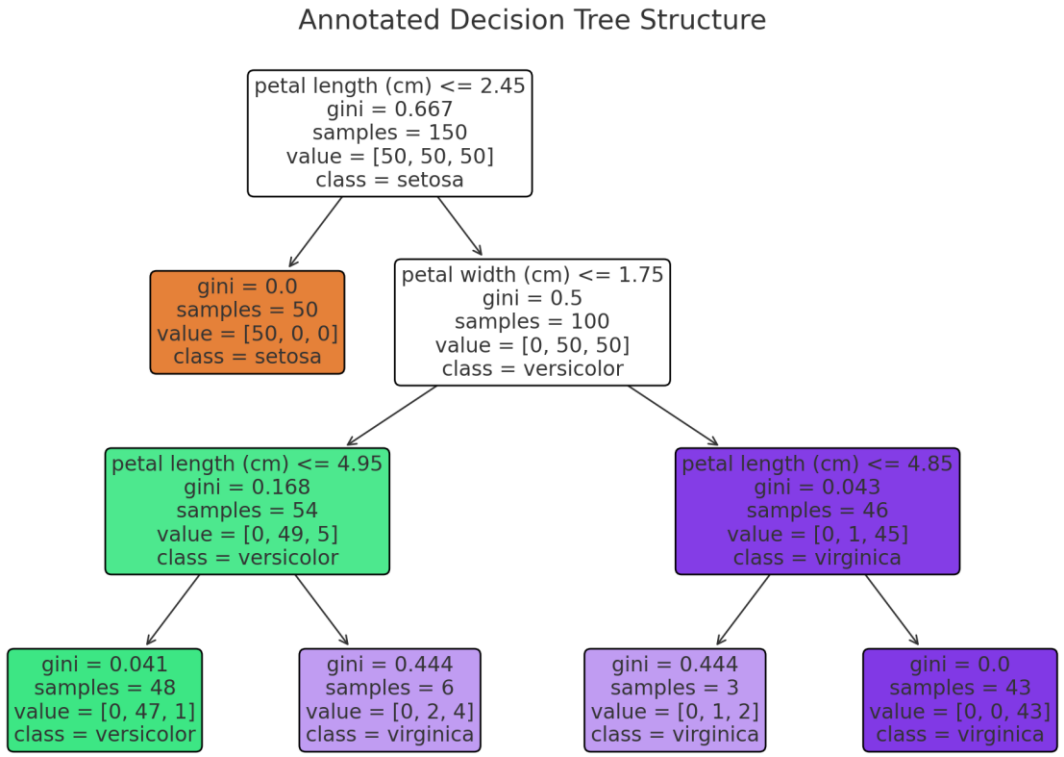
Control Methods:

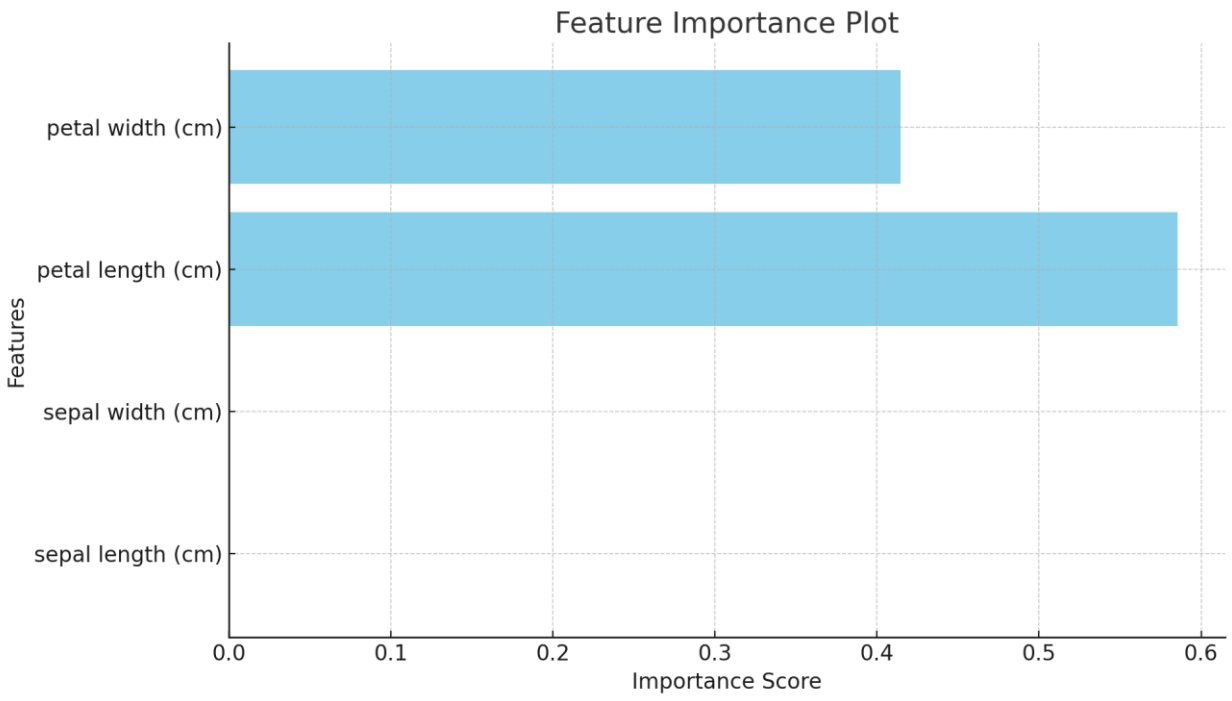
- **Pruning:** Reduces overfitting by removing branches with minimal information gain.
- **Max Depth:** Set a limit on tree depth to prevent excessive splits.
- **Minimum Samples per Leaf:** Require a minimum number of samples for each node split to ensure meaningful divisions.
- **Greedy Algorithms:** Decisions at each node are locally optimized to avoid exhaustive searching.

Explainability and Interpretability of Decision Trees

- **Interpretability:**
  Decision Trees are “white-box” models. Each split represents a step in decision-making, allowing users to trace the exact path leading to a prediction.
- **Readable Rules:**
  Each split in the tree forms an **if-then** rule. These rules break down complex decisions into clear, logical steps.
- **Visualization:**
  Tree diagrams visually illustrate decision paths and highlight influential features. This visual clarity aids in verifying the model’s logic.
- **Feature Importance:**
  Decision Trees provide **feature importance** scores, showing which features have the greatest impact on predictions. Features near the top of the tree usually hold more significance.

Decision Tree Explainability Illustration





Applications and Limitations of Decision Trees

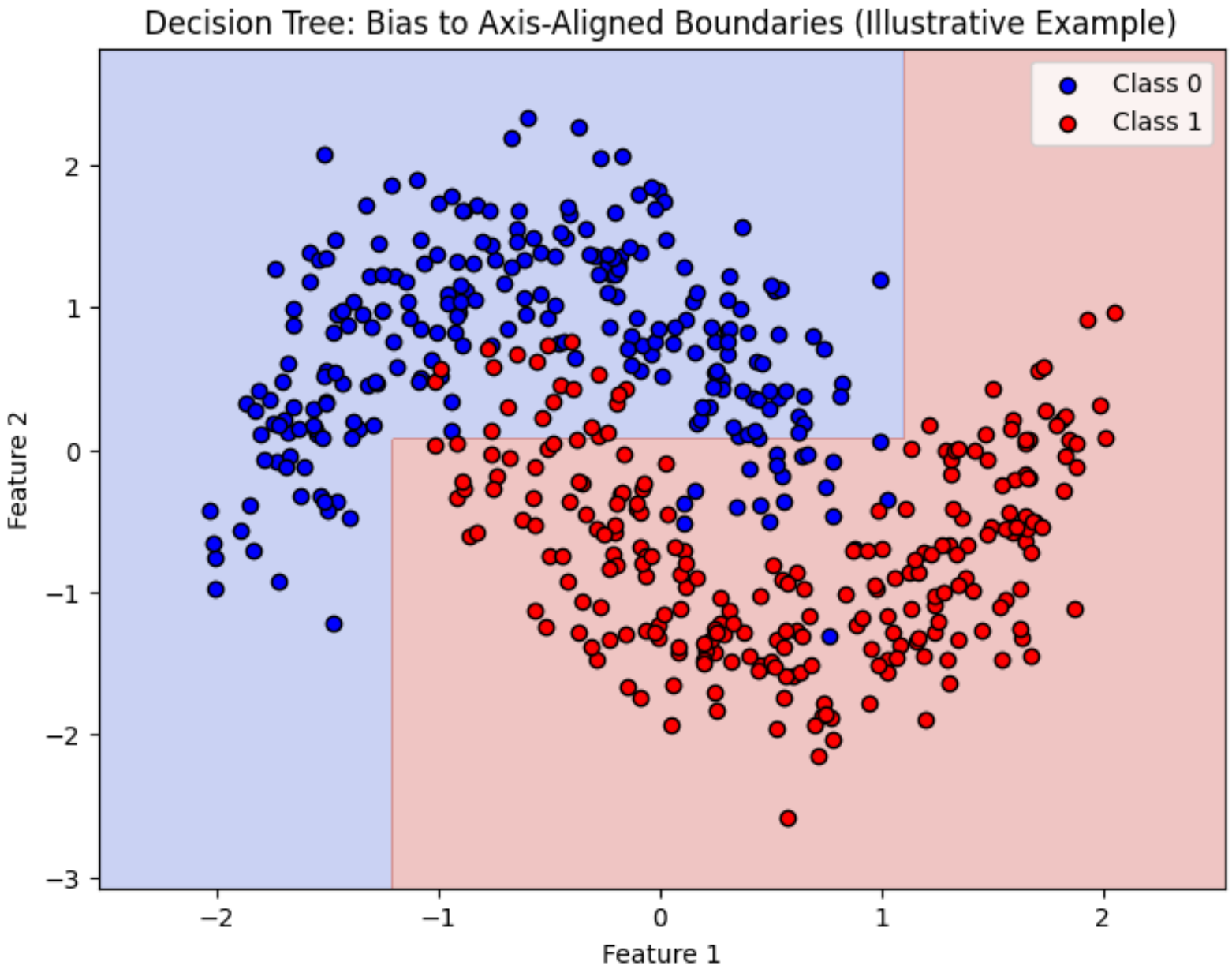
Applications

- 🩺 **Medical Diagnosis:** Classifying symptoms to diagnose diseases.
- 📊 **Customer Segmentation:** Dividing customers into meaningful groups.
- 💳 **Credit Scoring:** Assessing credit risk based on financial data.
- ⚠️ **Anomaly Detection:** Identifying Outliers.

Practical Limitations:

- 💥 **Overfitting:** Deep trees may fit noise.
- 🌪️ **Instability:** Small changes in data can lead to different splits.
- ➖ **Bias to Axis-Aligned Boundaries:** Less effective for data with non-rectangular class separations.

Bias to Axis-Aligned Boundaries:
Less effective for data with non-rectangular class separations.



Live Coding & Demos

Logistic Regression

- **Definition**: Logistic Regression is **a linear classifier** used **for binary outcomes**, modeling the probability that a given input belongs to a certain class.
- **Mathematical Foundation**:

$$P(Y = 1|X) = \sigma(wX + b) = \frac{1}{1 + e^{-(wX+b)}}$$

where **σ** is the **sigmoid function** mapping values to probabilities between 0 and 1.

Cost Function: Logistic loss (binary cross-entropy)

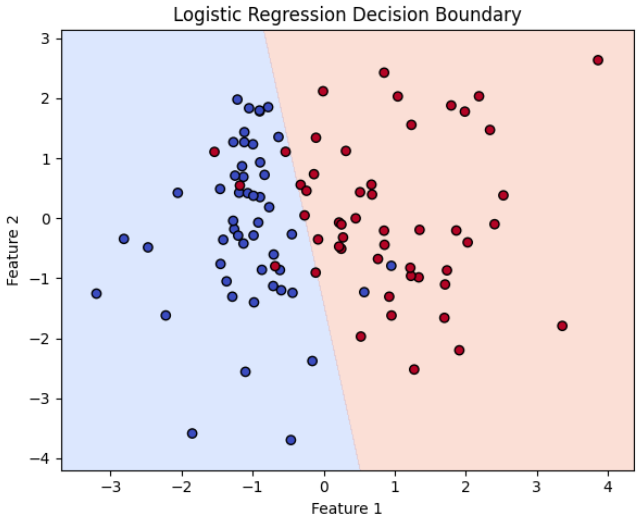
$$J(w, b) = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(\widehat{y_i}) + (1 - y_i)\log(1 - \widehat{y_i})]$$

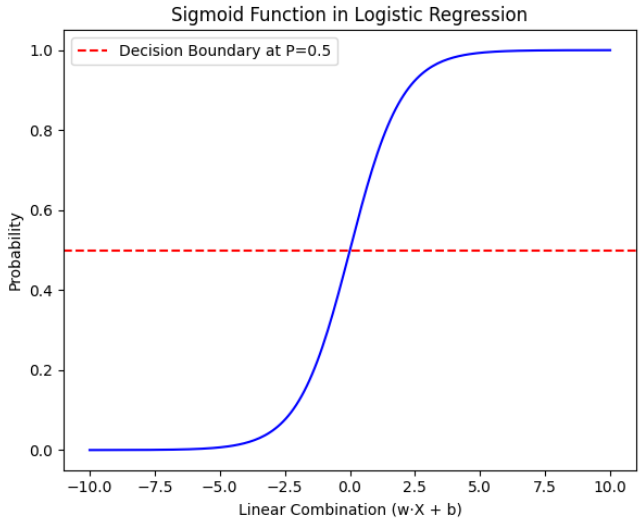
where $\widehat{y_i} = wX_i + b$ is the predicted prob for sample i

Decision Boundary for Logistic Regression:
The model forms a **linear decision boundary** at $P(Y = 1|X) = \tau$, which simplifies to $wX + b = 0$

$$\hat{Y} = \begin{cases} 1 \text{ if } P(Y = 1|X) \geq \tau \\ 0 \text{ if } P(Y = 1|X) \leq \tau \end{cases}$$

Usually, τ is set as 0.5





Requirements for Logistic Regression

- **Linearly Separable Data:** Performs best with linearly separable data or data close to being linearly separable.
- **Independence of Observations:** Assumes that each data point is independent.
- **No Multicollinearity:** Strongly correlated features can distort coefficient estimates, so collinearity should be minimized.
- **Absence of Outliers:** Outliers can disproportionately influence the model's parameters, leading to skewed results.
- **Feature Scaling Recommended:** Although not strictly required, scaling features improves convergence speed and stability.

Computational Complexity of Logistic Regression

Training Complexity: $O(mn)$; *m* - training samples; *n* - features. This complexity comes from the iterative process of adjusting weights for each feature across all samples.

Optimization Process:

- **Gradient Descent:** $O(mn)$; per iteration, ideal for large datasets.

$$\boxed{w = w + \alpha \frac{\partial l}{\partial w}; \quad b = b + \alpha \frac{\partial l}{\partial b}}$$

where α is the learning rate, which controls the step size of each update.

- **Newton-Raphson:** $O(mn^2 + n^3)$; per iteration, faster convergence but computationally expensive for high-dimensional data.

$$\boxed{w = w - H^{-1} \frac{\partial l}{\partial w}}$$

where H is the Hessian matrix, representing the second-order derivatives of ℓ with respect to w and b.

Prediction Complexity:

- $O(n)$ per sample, it requires calculating the dot product of the feature vector and coefficients.

Explainability of Logistic Regression

- **Interpretable Coefficients:** Each feature has a coefficient wj , indicating how it impacts the log-odds of the outcome.
  - **Positive** w_j : increases the probability of the positive class.
  - **Negative** w_j : decreases the probability of the positive class.

$$\boxed{w_j = \log\left(\frac{P(Y=1|X+\Delta X_j)}{P(Y=0|X+\Delta X_j)}\right)}$$

- **Odds Ratios:** By exponentiating coefficients, we obtain odds ratios e^{w_j}, showing how likely the outcome is with a one-unit increase in the feature.

- **Probability Estimates:** Logistic Regression outputs probabilities, allowing flexible decision thresholds based on application needs.

- **Simple Decision Boundary:** Creates a linear boundary, making the separation of classes easy to visualize and explain.

Logistic Regression Limitations

- **Limited to Binary Classification** (without adaptation): Applying it to multiclass problems requires extensions like One-vs-Rest or SoftMax regression.
- **Bias to Linear Boundaries:** Logistic Regression assumes that classes are linearly separable, using a straight line (or hyperplane) as the boundary.
- **Sensitivity to Outliers:** Outliers can disproportionately influence the decision boundary, especially in small datasets
- **Multicollinearity:** Highly correlated features can lead to unreliable coefficients.
- **Sensitivity to Imbalanced Classes:** Logistic Regression can struggle with imbalanced datasets where one class is much more prevalent.

Strengths and Applications of Logistic Regression

Strengths:

- **Interpretability:** Coefficients provide direct insight into feature importance.
- **Probability Estimates:** Outputs probabilities, which are useful for decision-making thresholds.
- **Computational Efficiency:** Fast to train and easy to implement for large datasets with low feature counts.

Applications:

- **Customer Churn Prediction:** Estimating the likelihood of customer leaving.
- **Medical Risk Prediction:** Calculating health risk probabilities based on patient data.
- **Credit Scoring:** Assessing loan default risk from financial features.

Live Coding & Demos

Support Vector Machine (SVM) – Support Vector Classification (SVC) overview

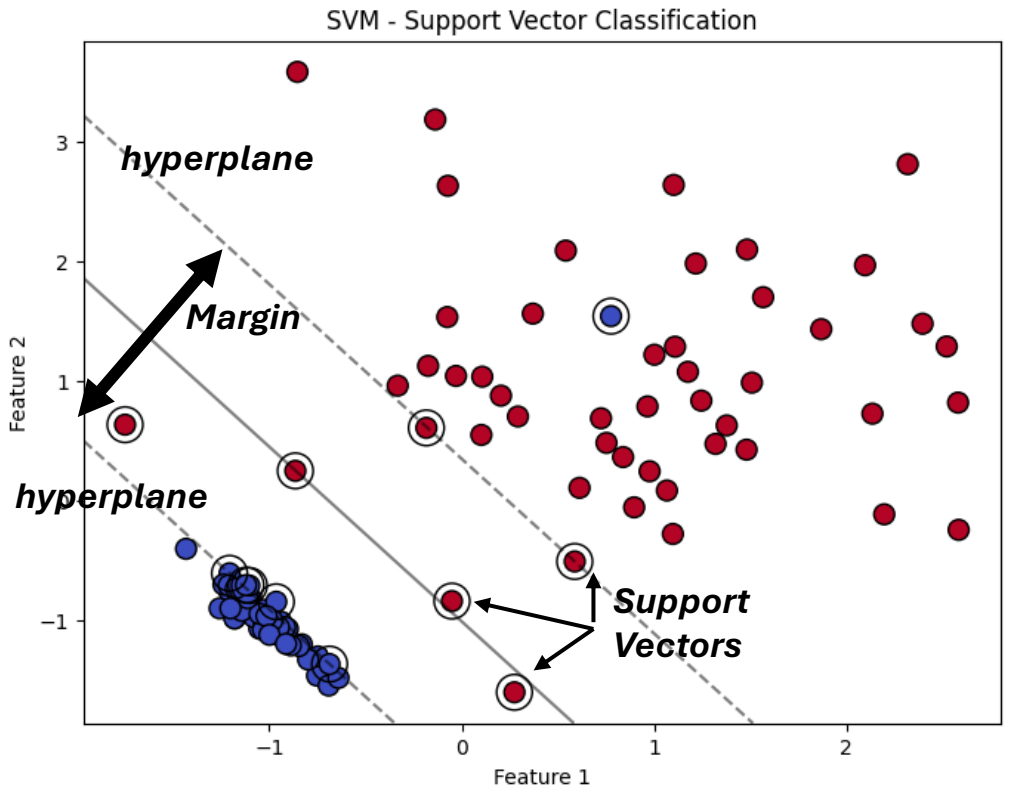
SVM-SVC is a supervised machine learning algorithm used for binary classification tasks.

- **Objective:** Identify a hyperplane $f(x) = wx + b$ that maximizes the margin (distance) between classes.

$$d = \frac{|wx + b|}{||w||}$$

For the support vectors, which lie on the boundary of the margin we have f(x)=±1

- **Decision Function:** For an input x, SVM predicts based on: $f(x) = wx + b$ where *w* is the weight vector and b is the bias.
- **Classification Rule:** $\hat{Y} = \begin{cases} 1 \text{ if } f(x) \geq 0 \\ 0 \text{ if } f(x) \leq 0 \end{cases}$



Cons:
- High computational cost
- Sensitive to parameter tuning
- Does not provide probability estimation

Pros:
- Effective in high dimensions
- Robust to outliers
- Adapts to non-linear data with kernels

Naïve Bayes - overview

Method Overview:
Naïve Bayes is a **probabilistic classifier** based on Bayes' Theorem, calculating the probability of each class given input features. **It assumes feature independence within each class**, simplifying computations.

Bayes' Theorem Formula: $P(C_k|X) = \frac{P(X|C_k)P(C_k)}{P(X)}$ where Ck is a class, and X is the input features.

Prediction Process:

- **Prior Probability:** Calculate $P(Ck)$ based on class frequencies.
- **Likelihood:** For each feature, compute $P(xi|Ck)$.
- **Combine with Independence Assumption:**

$$P(C_k|X) \propto P(C_k) \prod_{i=1}^{n} P(x_i|C_k)$$

- **Assign Class**: Choose the class with the highest posterior probability.

Types of Naïve Bayes:
•**Gaussian:** Assumes continuous data follows a normal distribution.
•**Multinomial:** For discrete counts, often used in text classification.
•**Bernoulli:** For binary features, useful in presence/absence tasks.

Model Evaluation

Classification Metrics - How to measure the results?

- Confusion Matrix:

		Predicted	
Actual		Positive	Negative
	Positive	**True Positive, TP**	**False Negative, FN**
	Negative	**False Positive, FP**	**True Negative, TN**

True Positive (TP): Correctly predicted positive cases.
False Positive (FP): Incorrectly predicted positive cases (Type I error).
True Negative (TN): Correctly predicted negative cases.
False Negative (FN): Incorrectly predicted negative cases (Type II error).

False Positive Rate (FPR): The proportion of actual negatives incorrectly classified as positive.

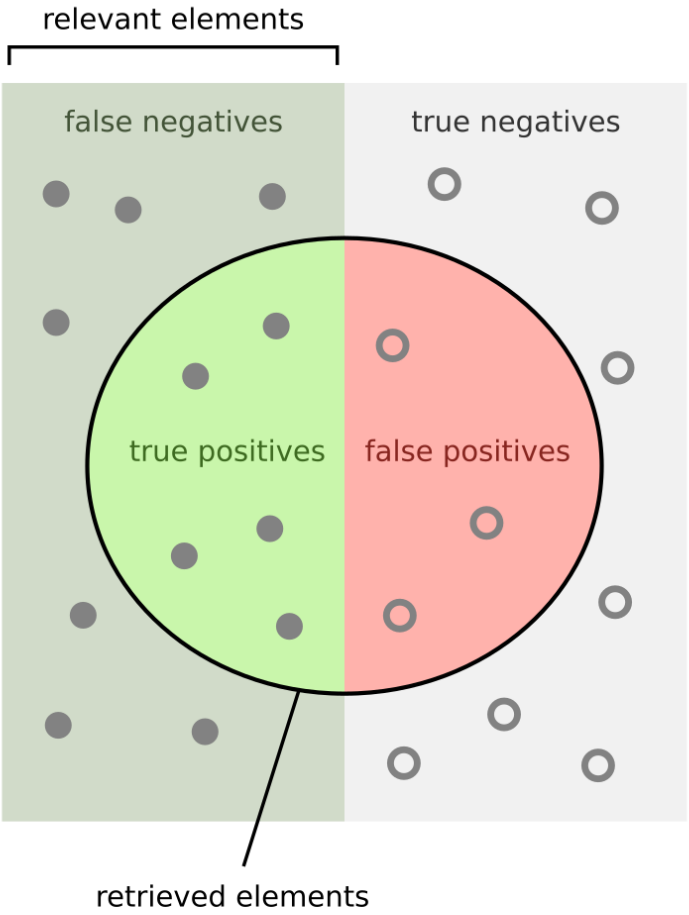
$$FPR = FP/(FP + TN)$$

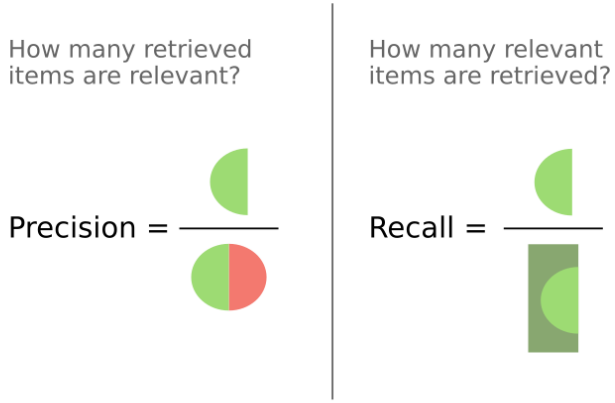
False Negative Rate (FNR): The proportion of actual positives incorrectly classified as negative.

$$FNR = FN/(FN + TP)$$

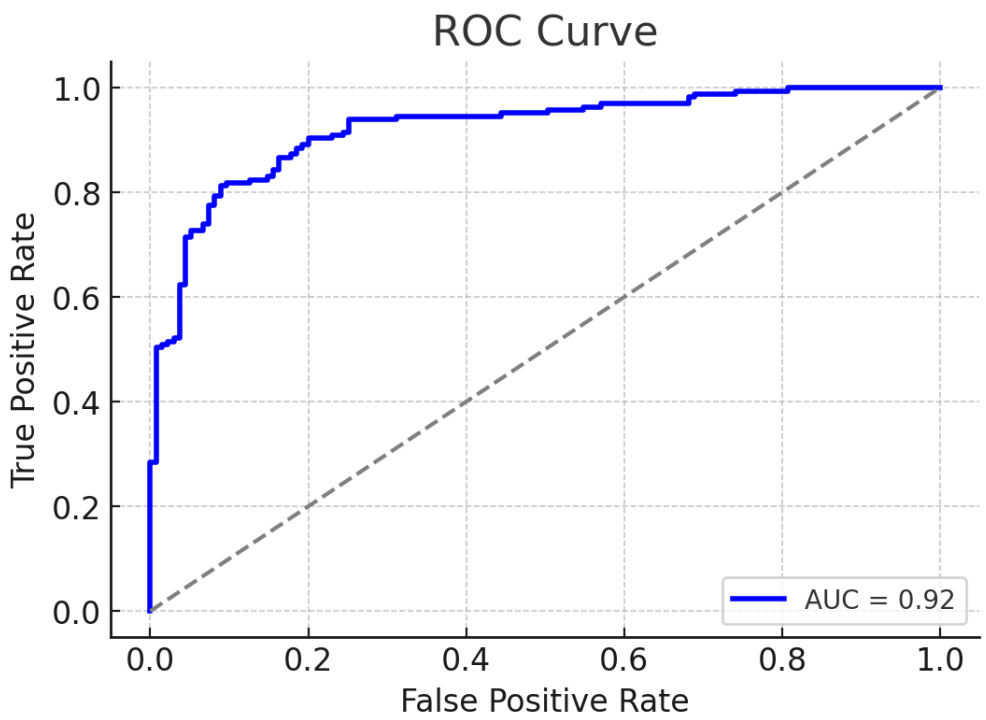
Basic Metrics: Accuracy, Precision, Recall, F-betta score

- **Accuracy:** The ratio of correct predictions to total predictions. $Accuracy = (TP + TN)/(TP + FP + FN + TN)$
  - Limitation: Can be misleading in imbalanced datasets.
- **Precision:** The ratio of true positives to all predicted positives. $Precision = TP/(TP + FP)$
  - Useful in cases where the cost of false positives is high
- **Recall (Sensitivity):** The ratio of true positives to all actual positives. $Recall = TP/(TP + FN)$
  - Important when missing a positive case has a high cost
- **F-Beta Score**: A weighted harmonic mean of Precision and Recall, with β determining the balance (e.g., F1 score with β=1 balances both equally).
- $F_\beta = (1 + \beta^2)\frac{Precision\ Recall}{\beta^2 Precision + Recall}$



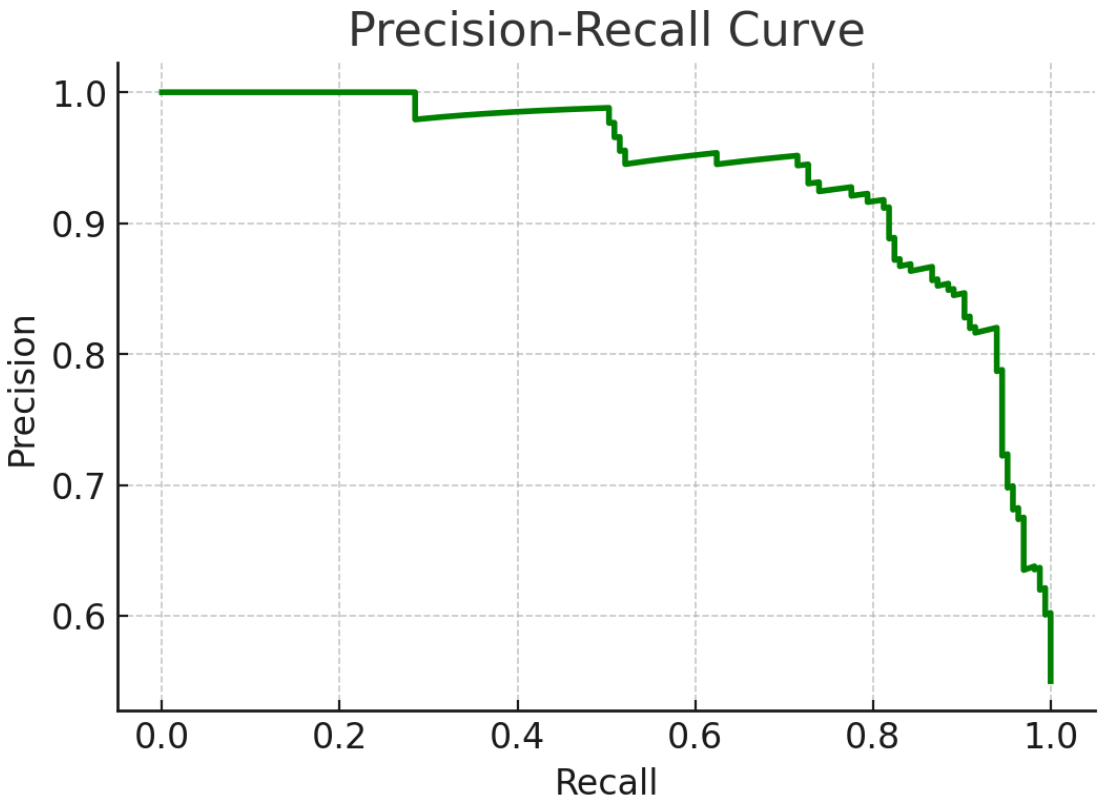


Advanced Metrics



The **AUC (Area Under Curve)** value here is 0.92, indicating strong model performance.

Gini Coefficient: $2AUC - 1$



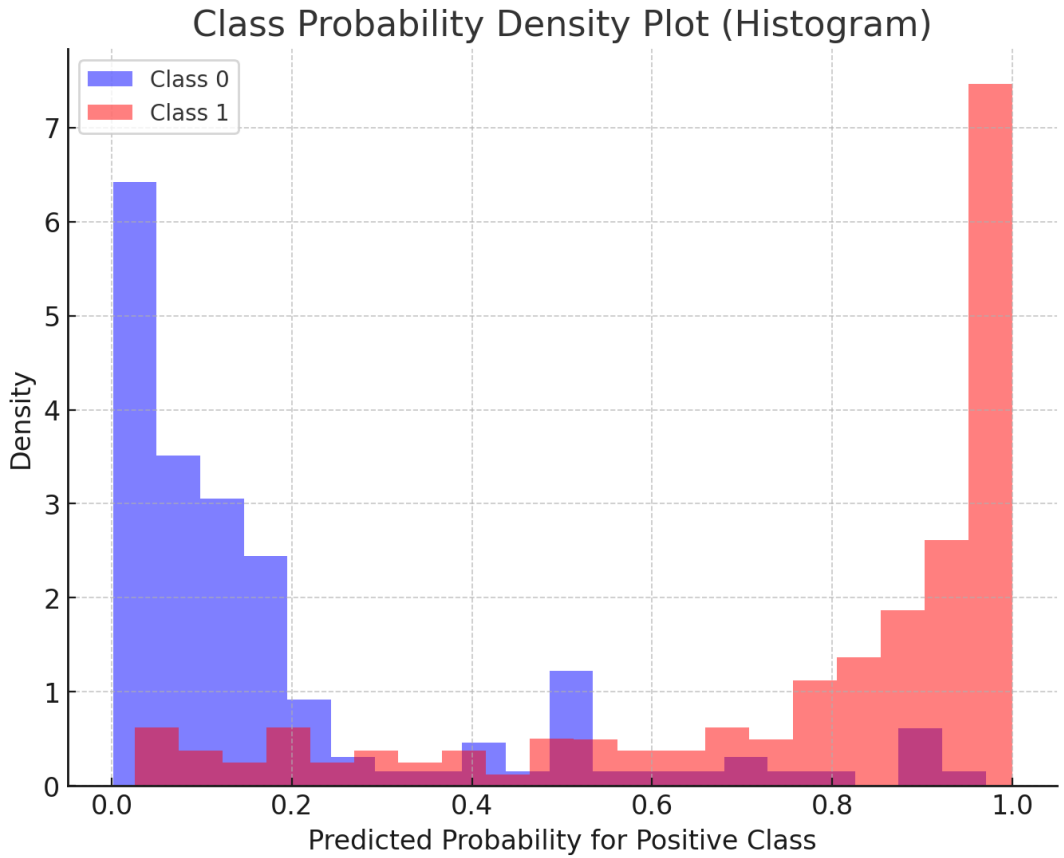
Useful when the classes are imbalanced.

Advanced Metrics

- **Logarithmic Loss (Log Loss)**: Evaluates the probability estimates of predictions, penalizing confident but incorrect predictions more harshly.

$$J(w,b) = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)]$$

- **Class Probability Density Plot:** A plot that shows the distribution of predicted probabilities for each class.
  - **Useful**: Helps visualize how well-separated the predicted probabilities are for each class, indicating model confidence and uncertainty.



Additional: https://neptune.ai/blog/evaluation-metrics-binary-classification

Live Coding & Demos

Challenges

Common Challenges for Classification Tasks

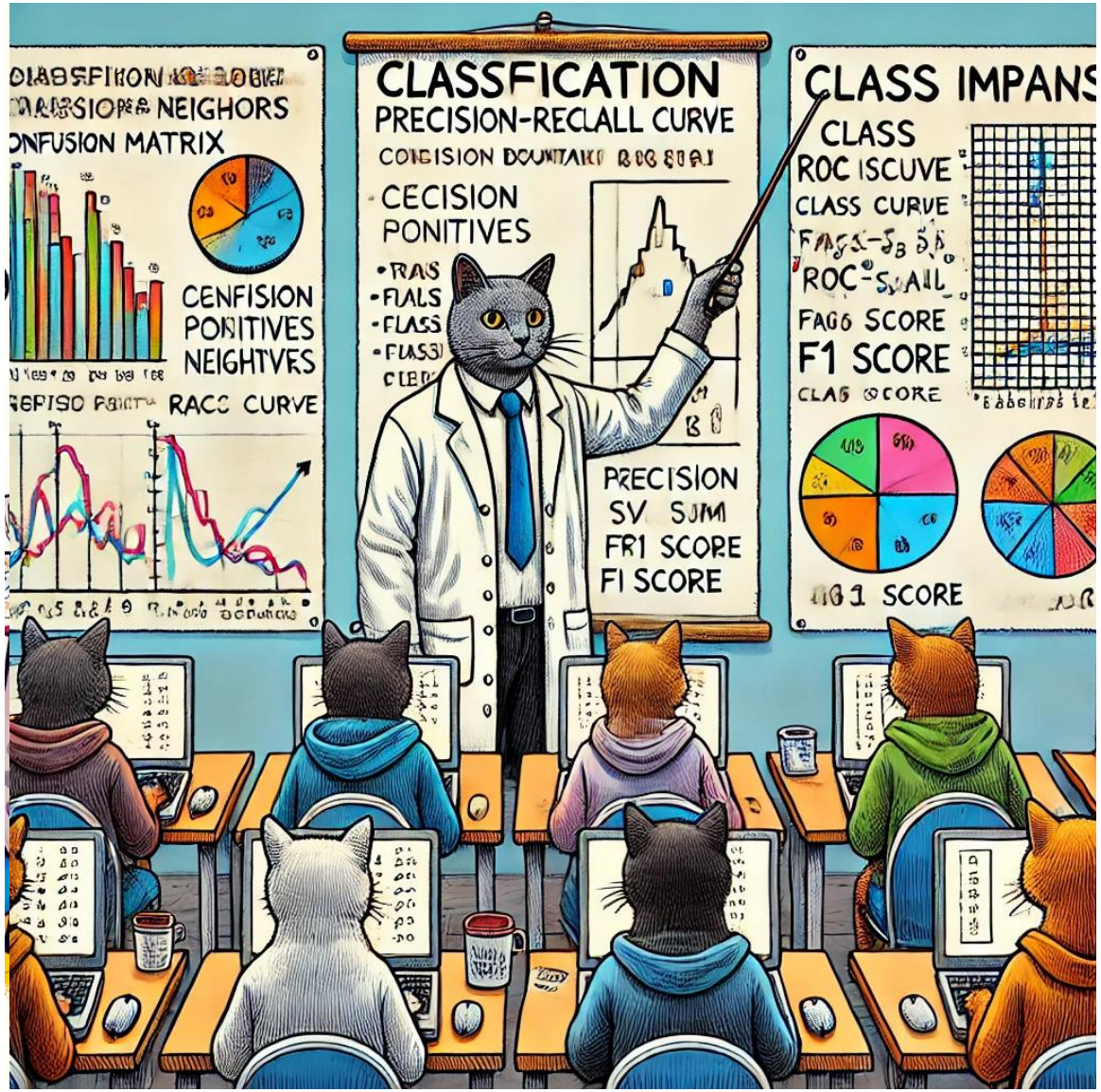
1. **Class Imbalance**:
    1. When classes are unevenly represented, leading to bias toward the majority class.
    2. **Solution**: Resampling techniques, SMOTE, or weighted loss functions.
2. **Multiclass & Multilabel**:
    1. **Multiclass**: More than two categories (e.g., cat, dog, bird).
    2. **Multilabel**: Multiple labels per instance (e.g., "indoor" and "nature").
    3. **Challenge**: Requires complex algorithms for overlapping labels.
3. **Ordinal Classes**:
    1. Ordered categories (e.g., low, medium, high) where distances aren't equal.
    2. **Solution**: Specialized models for ordinal data.
4. **High-Dimensional Data**:
    1. Many features increase computational cost and risk overfitting.
    2. **Solution**: Dimensionality reduction (e.g., PCA).
5. **Interpretability**:
    1. Complex models (e.g., neural networks) lack transparency.
    2. **Solution**: Interpretable models or explanation tools like SHAP, LIME.

Key Notes

Key Notes and Takeaways

- **Classification Task: Categorizing data,** essential for applications like spam detection and customer segmentation.
- **Techniques:** KNN, Decision Trees, Logistic Regression, SVM, Naïve Bayes—each suited to different data types.
- **Metrics:** Evaluate with Accuracy, Precision, Recall, AUC, PR Curve, and Log Loss helps measure effectiveness in various contexts.
- **Challenges:** Handle class **imbalance**, **high-dimensional data**, and **interpretability** with resampling, feature reduction, and tools.
- **Model Selection:** Balance accuracy and transparency; use simpler models for interpretability, or tools to explain complex ones..

Thank you for your attention



Appendix A

Contingency table

Contingency table

Sources: [1][2][3][4][5][6][7][8] view · talk · edit

		Predicted condition			
	Total population $= P + N$	**Predicted positive (PP)**	**Predicted negative (PN)**	Informedness, bookmaker informedness (BM) $= TPR + TNR - 1$	Prevalence threshold (PT) $= \frac{\sqrt{TPR \times FPR} - FPR}{TPR - FPR}$
Actual condition	**Positive (P)** [a]	**True positive** (TP), hit[b]	**False negative** (FN), miss, underestimation	True positive rate (TPR), recall, sensitivity (SEN), probability of detection, hit rate, power $= \frac{TP}{P} = 1 - FNR$	False negative rate (FNR), miss rate type II error [c] $= \frac{FN}{P} = 1 - TPR$
	Negative (N)[d]	**False positive** (FP), false alarm, overestimation	**True negative** (TN), correct rejection[e]	False positive rate (FPR), probability of false alarm, fall-out type I error [f] $= \frac{FP}{N} = 1 - TNR$	True negative rate (TNR), specificity (SPC), selectivity $= \frac{TN}{N} = 1 - FPR$
	Prevalence $= \frac{P}{P + N}$	Positive predictive value (PPV), precision $= \frac{TP}{PP} = 1 - FDR$	False omission rate (FOR) $= \frac{FN}{PN} = 1 - NPV$	Positive likelihood ratio (LR+) $= \frac{TPR}{FPR}$	Negative likelihood ratio (LR−) $= \frac{FNR}{TNR}$
	Accuracy (ACC) $= \frac{TP + TN}{P + N}$	False discovery rate (FDR) $= \frac{FP}{PP} = 1 - PPV$	Negative predictive value (NPV) $= \frac{TN}{PN} = 1 - FOR$	Markedness (MK), deltaP (Δp) $= PPV + NPV - 1$	Diagnostic odds ratio (DOR) $= \frac{LR+}{LR-}$
	Balanced accuracy (BA) $= \frac{TPR + TNR}{2}$	F_1 score $= \frac{2\,PPV \times TPR}{PPV + TPR} = \frac{2\,TP}{2\,TP + FP + FN}$	Fowlkes–Mallows index (FM) $= \sqrt{PPV \times TPR}$	Matthews correlation coefficient (MCC) $= \sqrt{TPR \times TNR \times PPV \times NPV} - \sqrt{FNR \times FPR \times FOR \times FDR}$	Threat score (TS), critical success index (CSI), Jaccard index $= \frac{TP}{TP + FN + FP}$

https://en.wikipedia.org/wiki/Evaluation_of_binary_classifiers